

a. Name of two GenAI programs

The 2 main AI's that I used to execute the programs was Gemini and ChatGPT,

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both of these LLMS through the main website for both Gemini and OpenAI websites.

c. Prompt used. (You must use the same prompt for both GenAIs.)

The prompt that was used for both of these AIs was as shown:

Objective

Write a C program that picks a secret number between 1 and 10.

The user gets up to 3 tries to guess the number.

Requirements

- Use a fixed secret number (for example, 7).
- Use printf to prompt the user and scanf to read their guess.
- Use a while or for loop to let the user guess up to 3 times.
- Use if-else statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Example Output

text

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

d. Generated code from both GenAIs.

Gemini Code:

```
#include <stdio.h>

int main(void) {
    const int secret_number = 7;
    const int max_attempts = 3;
    int guess;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= max_attempts; attempt++) {
        printf("Attempt %d/%d. Enter your guess: ", attempt,
max_attempts);
```

```

scanf("%d", &guess);

if (guess == secret_number) {
    printf("Correct! You win!\n");
    won = 1;
    break;
} else if (guess < secret_number) {
    if (attempt < max_attempts) {
        printf("Too low! Try again.\n");
    }
} else {
    if (attempt < max_attempts) {
        printf("Too high! Try again.\n");
    }
}

}

if (!won) {
    printf("You ran out of attempts! The secret number was %d. You
lose!\n", secret_number);
}

return 0;
}

```

Chat GPT Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int secretNumber = 7;
```

```
    int guess;
```

```
    int attempt = 1;
```

```
    int won = 0;
```

```
    printf("Guess a number between 1 and 10.\n");
```

```
    while (attempt <= 3) {
```

```
        printf("Attempt %d/3. Enter your guess: ", attempt);
```

```
        scanf("%d", &guess);
```

```
        if (guess == secretNumber) {
```

```
            printf("Correct! You win!\n");
```

```

        won = 1;
        break;
    } else if (guess < secretNumber) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too high! Try again.\n");
    }

    attempt++;
}

if (!won) {
    printf("Sorry, you lose! The secret number was %d.\n", secretNumber);
}

return 0;
}

```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Gemini Code:

Correctness: No errors while running, although if I type in a letter of any sorts, the program skips all 3 attempts while running so it doesn't have edge case protection(this also includes the fact that if I write a number over 10, it still counts it).

Execution time: The code consists of a singular for loop, which gives the program an $O(n)$ complexity since it's not a nested loop. Other than that, there aren't any functions within the loop that could cause the execution time to increase(the only function that could possibly cause issues is scanf, but that isn't inherently affecting the process of the loop, since it only scans for input that it's given).

Space Complexity: My main concern that this code has involving space complexity is the use of certain integers in the code such as the “int won”, which I feel might add unnecessary space complexity to the code since it isn't necessary to execute the code with and can be optimized for it to not be used within the program. Additionally, this code also has a sort of counter method which is used in the for loop to keep track of the attempts.

Maintainability: When it comes to maintainability, there are almost no comments on any of the lines, nor has the code been checked for any edge cases that cause the code to not work as needed. Instead, the code seems to unnecessarily return 0 at the very end of the code, which doesn't seem to be affecting anything in the actual code.

Chat GPT Code:

Correctness: No major errors while running. It still has the same issue as the gemini code where it can't handle major edge cases such as using letters for the input and doesn't have a bound

Execution Time: Execution time seems to be worse than the Gemini code, since it uses a while loop instead of a for loop, which is still $O(n)$ complexity, but is still less efficient for this prompt versus the for loop.

Space Complexity: Also has an unnecessary int of "won", which also doesn't seem to be an efficient way of closing out the code and could be made more efficiently. Chat GPT also seems to return 0 at the end of the code as well, which might be wasting some extra space while running the code.

Maintainability: This code also has completely no comments on any of the lines, and hasn't checked for any of the edge cases that could easily occur while running this program.

f. Select one to use as the basis for this assignment and justify why you picked it.

For my code, I decided to use the Gemini code as the base because the general structure of the code using a for loop is way more efficient than ChatGPTs while loop, so I can work with that and modify the code as needed so that it becomes a lot more efficient.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness:

I cleaned up any possible errors that could occur when it comes to the edge cases, so it should only take in an answer that works for the given prompt. I also made it so that the program doesn't count an invalid input as an attempt.

Execution Time: I made the return happen early on in the code instead of the (int win = 0) that was used in the original code to allow for less Execution time. Despite this, I did add a while loop that checks for incorrect inputs, but it's only looking through a single input given by the user.

Space Complexity: I got rid of the int won from the original code, since it made the code run longer than it needs to so I modified it so it would work around using an extra constant integer.

Maintainability: I put in all of the comments and prologue in order to give an in-depth description of how the code works. I've also error - checked all of the code as well so none of the edge cases should cause an error in the execution.